

Software Engineering Assignment #2

AI Analysis

Access and Prompt Method of AI Used

For this assignment, I used **Gemini** and **ChatGPT**. Both models I accessed using their **website versions**. For Gemini, I used its **3.1 Pro model with extended thinking** for my prompt. And for ChatGPT, I toggled its **think mode**. Both models were accessed using a free account. The prompt I gave both AIs follows:

Program Overview

The program is a C program that takes in a file that has a list of commands. The program will handle these commands and print the output to the user's console.

Implementation

The program will define its own MaxHeap. The MaxHeap must have a list-based implementation that creates a priority queue. No pre-existing heap functions or modules can be used.

Email Definition and Priority

The MaxHeap should expect to take in an Email. An Email is an object with a category, subject, and date. The category field can only be "Boss", "Subordinate", "Peer", "ImportantPerson", and "OtherPerson". The MaxHeap should first try and sort the email by these categories with "Boss" being the most prioritized and "OtherPerson" being the least. In the event of a conflict, the date field should be used to place the email. The date field is defined in the format "MM-DD-YYYY" and is a string. The newest email will be prioritized over the older email in the event that they share the same category. The subject field is a string.

Commands

There are 4 commands that the program must handle.

EMAIL <sender_category>,<subject_line>,<date>	The email command will add an email to the MaxHeap adhering to the priority information in the previous section.
NEXT	Displays information for next email in MaxHeap in format: "Sender: Subject: Date:"
READ	Removes email from MaxHeap
COUNT	Displays count of unread emails

Input

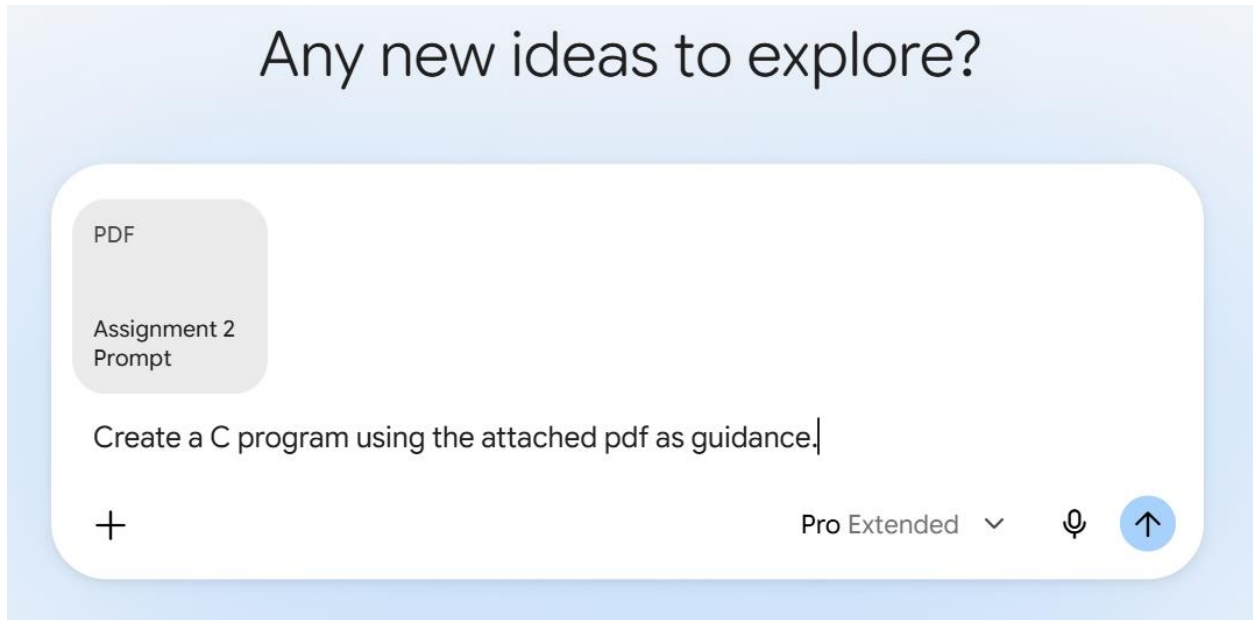
The program assumes that the format of the test file adheres to the format declared above but does not assume that the commands are not nonsensical. There are a few rules that the program must follow:

- NEXT or READ are invalid if there are no emails in the MaxHeap
- NEXT is invalid without a READ command between
- READ is invalid without a NEXT command between

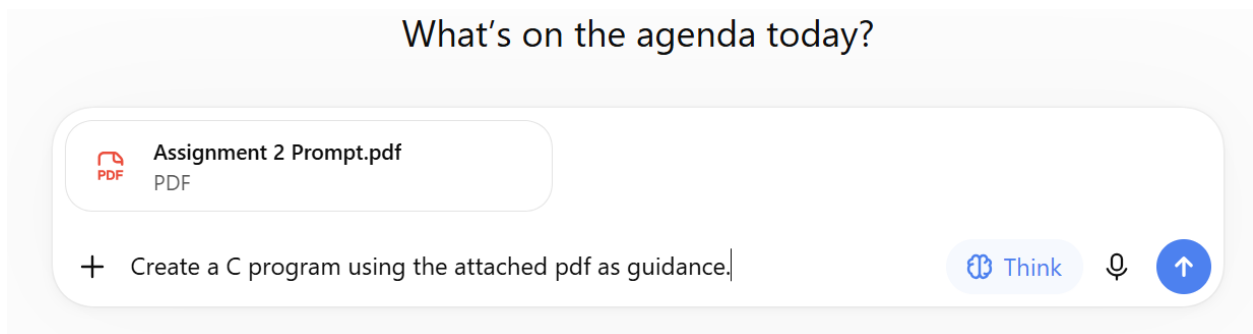
Example Run

- Sample Test File:
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
- Output of Sample Test File:

There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.



(Fig. 1 – Gemini Setup)



(Fig 2. - ChatGPT Setup)

AI Analysis

Gemini Output

```
#include <stdio.h> #include <stdlib.h> #include <string.h> #include <ctype.h>

#define MAX_EMAILS 1000 #define MAX_STR_LEN 256

// Represents a single email as outlined in the assignment criteria typedef struct { char
category[MAX_STR_LEN]; char subject[MAX_STR_LEN]; char date[MAX_STR_LEN]; }
Email;

// MaxHeap definition typedef struct { Email data[MAX_EMAILS]; int size; } MaxHeap;

// Utility function to trim leading and trailing whitespaces char *trimWhitespace(char
*str) { char *end; while(isspace((unsigned char)*str)) str++; if(*str == 0) return str; end =
str + strlen(str) - 1; while(end > str && isspace((unsigned char)*end)) end--; end[1] = '\0';
return str; }

// Map the string category to an integer priority int getCategoryValue(const char* cat) { if
(strcmp(cat, "Boss") == 0) return 5; // Handle potential variations in spacing as seen in
the prompt vs example if (strcmp(cat, "Important Person") == 0 || strcmp(cat,
"ImportantPerson") == 0) return 4; if (strcmp(cat, "Subordinate") == 0) return 3; if
(strcmp(cat, "Peer") == 0) return 2; return 1; // "Other Person" or "OtherPerson" }

// Compare two emails based on Category, then by Date (Newest first) int
compareEmails(Email a, Email b) { int catA = getCategoryValue(a.category); int catB =
getCategoryValue(b.category);

// Category priority first
if (catA != catB) {
    return catA - catB;
}

// Tie-breaker: Date in MM-DD-YYYY format
int m1, d1, y1, m2, d2, y2;
sscanf(a.date, "%d-%d-%d", &m1, &d1, &y1);
sscanf(b.date, "%d-%d-%d", &m2, &d2, &y2);

// Convert to comparable integers (YYYYMMDD)
int date1 = y1 * 10000 + m1 * 100 + d1;
int date2 = y2 * 10000 + m2 * 100 + d2;

return date1 - date2;
```

```

}

// Swap two emails in the heap void swap(Email *a, Email *b) { Email temp = *a; *a = *b;
*b = temp; }

// Bubble up logic to maintain MaxHeap property after insertion void bubbleUp(MaxHeap
*heap, int index) { if (index == 0) return; int parent = (index - 1) / 2; if
(compareEmails(heap->data[index], heap->data[parent]) > 0) { swap(&heap-
>data[index], &heap->data[parent]); bubbleUp(heap, parent); }}

// Bubble down logic to maintain MaxHeap property after extraction void
bubbleDown(MaxHeap *heap, int index) { int leftChild = 2 * index + 1; int rightChild = 2 *
index + 2; int maxIndex = index;

if (leftChild < heap->size && compareEmails(heap->data[leftChild],
heap->data[maxIndex]) > 0) {
    maxIndex = leftChild;
}
if (rightChild < heap->size && compareEmails(heap->data[rightChild],
heap->data[maxIndex]) > 0) {
    maxIndex = rightChild;
}
if (maxIndex != index) {
    swap(&heap->data[index], &heap->data[maxIndex]);
    bubbleDown(heap, maxIndex);
}

}

// Insert an email into the heap void insertEmail(MaxHeap *heap, Email email) { if (heap-
>size >= MAX_EMAILS) return; // Prevent overflow heap->data[heap->size] = email;
bubbleUp(heap, heap->size); heap->size++; }

// Remove and return the top email from the heap Email extractMax(MaxHeap *heap)
{ Email maxEmail = heap->data[0]; heap->data[0] = heap->data[heap->size - 1]; heap-
>size--; bubbleDown(heap, 0); return maxEmail; }

int main(int argc, char *argv[]) { if (argc < 2) { printf("Usage: %s \n", argv[0]); return 1; }

FILE *file = fopen(argv[1], "r");
if (!file) {
    printf("Error: Could not open file %s\n", argv[1]);
    return 1;
}
}

```

```

MaxHeap heap;
heap.size = 0;

char line[512];

// State machine flags for NEXT and READ rules
int can_next = 1;
int can_read = 0;

while (fgets(line, sizeof(line), file)) {
    char *trimmedLine = trimWhitespace(line);
    if (strlen(trimmedLine) == 0) continue;

    if (strncmp(trimmedLine, "EMAIL ", 6) == 0) {
        // Parse EMAIL command
        char *category = strtok(trimmedLine + 6, ",");
        char *subject = strtok(NULL, ",");
        char *date = strtok(NULL, ""); // Read till end

        if (category && subject && date) {
            Email newEmail;
            strcpy(newEmail.category, trimWhitespace(category));
            strcpy(newEmail.subject, trimWhitespace(subject));
            strcpy(newEmail.date, trimWhitespace(date));
            insertEmail(&heap, newEmail);
        }
    }
    else if (strcmp(trimmedLine, "COUNT") == 0) {
        printf("There are %d emails to read.\n", heap.size);
    }
    else if (strcmp(trimmedLine, "NEXT") == 0) {
        if (heap.size == 0) {
            printf("Error: No emails in the MaxHeap.\n");
        } else if (!can_next) {
            printf("Error: NEXT is invalid without a READ command
between.\n");
        } else {
            printf("Next email:\n");
            printf("Sender: %s\n", heap.data[0].category);
            printf("Subject: %s\n", heap.data[0].subject);
            printf("Date: %s\n", heap.data[0].date);

            can_next = 0;
            can_read = 1;
        }
    }
}

```

```

    }
}
else if (strcmp(trimmedLine, "READ") == 0) {
    if (heap.size == 0) {
        printf("Error: No emails in the MaxHeap.\n");
    } else if (!can_read) {
        printf("Error: READ is invalid without a NEXT command
between.\n");
    } else {
        extractMax(&heap);
        can_read = 0;
        can_next = 1;
    }
}
}

fclose(file);
return 0;

}

```

ChatGPT

```

#include <stdio.h> #include <stdlib.h> #include <string.h> #include <ctype.h>

#define INITIAL_CAPACITY 10 #define MAX_LINE 1024

/*
    • Email definition:
    • category: Boss, Subordinate, Peer, ImportantPerson, OtherPerson
    • subject: string
    • date: MM-DD-YYYY

*/ typedef struct { char *category; char *subject; char *date; unsigned long long
sequence; } Email;

/* Array-based MaxHeap implemented from scratch. */ typedef struct { Email *items; int
size; int capacity; } MaxHeap;

/* ----- Utility functions ----- */

```

```

static char *duplicate_string(const char *source){ size_t length = strlen(source); char
*copy = malloc(length + 1);

if (copy == NULL) {
    fprintf(stderr, "Memory allocation failed.\n");
    exit(EXIT_FAILURE);
}

strcpy(copy, source);
return copy;

}

static int category_priority(const char *category){ if (strcmp(category, "Boss") == 0)
{ return 5; } if (strcmp(category, "Subordinate") == 0) { return 4; } if (strcmp(category,
"Peer") == 0) { return 3; } if (strcmp(category, "ImportantPerson") == 0) { return 2; } if
(strcmp(category, "OtherPerson") == 0) { return 1; }

return 0; /* Input is assumed valid according to the prompt. */

}

/* Converts MM-DD-YYYY into YYYYMMDD for easy chronological comparison. */ static
long date_value(const char *date){ int month, day, year;

if (sscanf(date, "%2d-%2d-%4d", &month, &day, &year) != 3) {
    return 0;
}

return (long)year * 10000L + month * 100L + day;

}

/*
    • Returns > 0 when a should have higher priority than b.
    • Priority rules:
    •
        ○ Category
    •
        ○ Newest date when category is the same
    •
        ○ Earlier insertion sequence as a deterministic tie-breaker

```

```

*/ static int higher_priority(const Email *a, const Email *b) { int category_a =
category_priority(a->category); int category_b = category_priority(b->category);

if (category_a != category_b) {
    return category_a > category_b;
}

if (date_value(a->date) != date_value(b->date)) {
    return date_value(a->date) > date_value(b->date);
}

return a->sequence < b->sequence;

}

static void free_email(Email *email) { free(email->category); free(email->subject);
free(email->date);

email->category = NULL;
email->subject = NULL;
email->date = NULL;

}

/* ----- MaxHeap functions ----- */

static void heap_init(MaxHeap *heap) { heap->items = malloc(INITIAL_CAPACITY *
sizeof(Email));

if (heap->items == NULL) {
    fprintf(stderr, "Memory allocation failed.\n");
    exit(EXIT_FAILURE);
}

heap->size = 0;
heap->capacity = INITIAL_CAPACITY;

}

static void heap_resize(MaxHeap *heap) { int new_capacity = heap->capacity * 2; Email
*new_items = realloc(heap->items, new_capacity * sizeof(Email));

```

```

if (new_items == NULL) {
    fprintf(stderr, "Memory allocation failed.\n");
    exit(EXIT_FAILURE);
}

heap->items = new_items;
heap->capacity = new_capacity;

}

static void swap_email(Email *a, Email *b) { Email temp = *a; *a = *b; *b = temp; }

static void heapify_up(MaxHeap *heap, int index) { while (index > 0) { int parent = (index -
1) / 2;

    if (!higher_priority(&heap->items[index], &heap->items[parent])) {
        break;
    }

    swap_email(&heap->items[index], &heap->items[parent]);
    index = parent;
}

}

static void heapify_down(MaxHeap *heap, int index) { while (1) { int left = index * 2 + 1; int
right = index * 2 + 2; int highest = index;

    if (left < heap->size &&
        higher_priority(&heap->items[left], &heap->items[highest])) {
        highest = left;
    }

    if (right < heap->size &&
        higher_priority(&heap->items[right], &heap->items[highest]))
{
        highest = right;
    }

    if (highest == index) {
        break;
    }

    swap_email(&heap->items[index], &heap->items[highest]);
}

```

```
        index = highest;
    }

}

static void heap_insert(MaxHeap *heap, Email email) { if (heap->size == heap->capacity)
{ heap_resize(heap); }

heap->items[heap->size] = email;
heapify_up(heap, heap->size);
heap->size++;

}

static Email heap_peek(const MaxHeap *heap) { return heap->items[0]; }

static Email heap_remove_max(MaxHeap *heap) { Email max_email = heap->items[0];

heap->size--;

if (heap->size > 0) {
    heap->items[0] = heap->items[heap->size];
    heapify_down(heap, 0);
}

return max_email;

}

static void heap_destroy(MaxHeap *heap) { int i;

for (i = 0; i < heap->size; i++) {
    free_email(&heap->items[i]);
}

free(heap->items);
heap->items = NULL;
heap->size = 0;
heap->capacity = 0;

}
```

```

/* ----- Command processing ----- */

static void trim_newline(char *line) { size_t length = strlen(line);

while (length > 0 && (line[length - 1] == '\n' || line[length - 1] ==
'\r')) {
    line[length - 1] = '\0';
    length--;
}

}

static void process_email_command(MaxHeap *heap, const char *arguments, unsigned
long long *sequence_counter) { char buffer[MAX_LINE]; char *category; char *subject;
char *date; Email email;

strncpy(buffer, arguments, sizeof(buffer) - 1);
buffer[sizeof(buffer) - 1] = '\0';

category = strtok(buffer, ",");
subject = strtok(NULL, ",");
date = strtok(NULL, ",");

if (category == NULL || subject == NULL || date == NULL) {
    return;
}

email.category = duplicate_string(category);
email.subject = duplicate_string(subject);
email.date = duplicate_string(date);
email.sequence = (*sequence_counter)++;

heap_insert(heap, email);

}

int main(int argc, char *argv[]) { FILE *input; char line[MAX_LINE]; MaxHeap heap;
unsigned long long sequence_counter = 0; int next_pending = 0;

if (argc != 2) {
    fprintf(stderr, "Usage: %s <input_file>\n", argv[0]);
    return EXIT_FAILURE;
}

```

```

input = fopen(argv[1], "r");
if (input == NULL) {
    perror("Could not open input file");
    return EXIT_FAILURE;
}

heap_init(&heap);

while (fgets(line, sizeof(line), input) != NULL) {
    char *command;
    char *arguments;

    trim_newline(line);

    if (line[0] == '\\0') {
        continue;
    }

    command = strtok(line, " ");
    arguments = strtok(NULL, "");

    if (command == NULL) {
        continue;
    }

    if (strcmp(command, "EMAIL") == 0) {
        if (arguments != NULL) {
            process_email_command(&heap, arguments,
&sequence_counter);
        }
    }
    else if (strcmp(command, "COUNT") == 0) {
        printf("There are %d emails to read.\n", heap.size);
    }
    else if (strcmp(command, "NEXT") == 0) {
        /* NEXT is invalid if there are no emails or if the previous
NEXT
        has not yet been followed by READ. */
        if (heap.size == 0 || next_pending) {
            continue;
        }

        Email next_email = heap_peek(&heap);

        printf("Next email:\n");

```

```

        printf("Sender: %s\n", next_email.category);
        printf("Subject: %s\n", next_email.subject);
        printf("Date: %s\n", next_email.date);

        next_pending = 1;
    }
    else if (strcmp(command, "READ") == 0) {
        /* READ is invalid if there are no emails or if there has not
           been a NEXT command since the previous READ. */
        if (heap.size == 0 || !next_pending) {
            continue;
        }

        Email read_email = heap_remove_max(&heap);
        free_email(&read_email);
        next_pending = 0;
    }
    /* Any other command is ignored. The prompt says the input
    commands
       may be nonsensical, so invalid commands do not produce output.
    */
}

heap_destroy(&heap);
fclose(input);

return EXIT_SUCCESS;

}

```

Correctness

List Based Heap Implementation

The requirements of the program state that a heap must be used to organize the emails. Not only that, but the heap itself has to be implemented using a list-based implementation. For the ChatGPT output, this heap has a variable "items" that points to an email (22-27). For the Gemini output, the heap definition has a variable "data" that is an array of emails with length "MAX_EMAILS" (16-20). While arrays and pointers are closely related and both programs use the concepts of a list-based implementation, an array is more clearly related to list-based representations.

Heap Functions

This section will look at if the functions of the heap work as intended. This will be evaluated by function and program. After each is assessed then this section will end in a conclusion of heap functionality for both programs.

- Gemini
 - swap (65-70)
 - Successfully switches the memory address at a to point at b and b to point at a
 - bubbleUp (72-80)
 - Performs recursive bubble up logic but does not check that index is valid for the heap
 - bubbleDown (82-98)
 - Successfully recursively bubbles down but does not check that index is valid for the heap
 - insertEmail (100-106)
 - Successfully inserts a new email into the heap but fails to inform the caller if overflow occurs
 - extractMax (108-115)
 - Successfully extracts email except for cases where the heap is empty
- ChatGPT
 - heap_init (109-119)
 - Allocates enough space for the heap at the starting capacity, does have protection if the memory fails to allocate.
 - heap_resize (121-132)
 - Doubles heap size successfully and has protection if the memory fails to allocate.
 - swap_email (134-138)
 - Successfully swaps the memory of emails at memory location a and b
 - heapify_up (140-151)
 - Does a successful enumerating heapify up but does not check that the index is actually in range
 - heapify_down (153-176)
 - Does a successful infinite while loop to heapify down but does not check that the index is valid
 - heap_insert (178-186)
 - Adds a new email to the heap successfully
 - heap_peek (188-190)

- Returns first element of heap successfully but does not check if heap is empty or not
- heap_remove_max (192-203)
 - Removes heap's top element and heapifies down the smallest element. Checks if heap becomes empty after removal but does not check if empty before removal.
- heap_destroy (205-216)
 - Removes all items in heap and clears memory.

Overall, both programs struggle with proper checks to prevent misinput. While these functions are internal, these errors make the program not as correct since it means that the program is not using a correct MaxHeap list-based implementation in its design.

Email Comparison

Both programs successfully handle comparison by first looking at email date but there is an issue in the comparison of categories. Gemini has ImportantPerson at a priority of 4 which should be much lower. ChatGPT has the correct categorization. However, there is a far more interesting comparison that occurs if emails are identical. In Gemini's comparison, it first checks if the categories are different. If they are, then Gemini return the result of the categories integer difference. If Gemini finds that the categories are the same, then it will convert the date string into a comparable integer format and then return the difference (43-63). ChatGPT does the exact same steps but after getting the date integers, it checks if they are equal. If the integers are the same, then it will return which email comes first (82-94). This has an important effect when emails have the same category and date. ChatGPT will order them in the order that the emails come in. So, I sent equal emails of 1,2,3,4,5, and 6. ChatGPT will return them in the order 1,2,3,4,5,6. On the other hand, Gemini will return that same list as 1,6,5,4,3,2. The reason for this shift is that a key part of the MaxHeap is when the top element is removed. In a standard MaxHeap, the maximum value is picked and then heapified down to keep the MaxHeap adhering to the sorting algorithm. For both programs given a list of 6 exact emails, the initial order of the MaxHeap would be 1,2,3,4,5,6. Once the first element is removed, the MaxHeap goes to correct by making the list 6,2,3,4,5. At this point, the heap tries to heapify down the 6th email. ChatGPT finds that the 6th email came after emails 2 and 3 and swaps with 2 to keep the MaxHeap in order before looping until creating a MaxHeap with element 2,3,4,5,6. Gemini on the other hand simply finds that the dates are equal and so doesn't heapify at all and leaves the list at 6,2,3,4,5. Simply put, Gemini has no tiebreaker whereas ChatGPT decides by which email came first. ChatGPT is more correct than Gemini in this instance since it keeps a consistent order but it is still incorrect according to the instructions. On sorting,

the instructions say: “If there is more than one from a sender, then the newest email (not the oldest) should be read first.” which contradicts the ChatGPT sorting. The most accurate sorting would use which email was instantiated last as the maximum not as the minimum. Therefore, both of these programs are wrong in their own way, but ChatGPT is slightly more correct than Gemini since it is consistent.

Output Format

Both programs have the same issue with formatting. They do not put lines between command outputs and lack any tabs. Instead of:

You have 6 unread emails

Next email:

Sender: Peer

Subject: Correct input

Date: 12-05-2005

The programs output:

You have 6 unread emails

Next email:

Sender: Peer

Subject: Correct input

Date: 12-05-2005

The programs are incorrect because of this since they should match the format of the output exactly. However, I will note that the prompt file I provided was incorrect on this as well so that is likely why they produce these results.

File Handling

Both programs were capable of handling the sample input provided in the instructions. However they showed different behavior in edge cases. For this section I will list the different edge cases and examine each one separately. However, I will be skipping the analysis of the base input since both performed flawlessly and the analysis of emails with exact same category and date since that has been covered.

- Surprise Email
 - This section will focus on a situation where the CEO has pulled up an email but an email with higher priority has come in. This tests if the program can properly handle the next and read tasks when the situation changes between operations.
 - Example used
 - EMAIL OtherPerson,You have a terminal illness,12-05-2001
 - COUNT
 - NEXT
 - EMAIL Boss,Andy Reid is stopping by,12-05-2005
 - READ
 - COUNT
 - NEXT
 - READ
 - COUNT
 - Both programs fail this test since the NEXTs both show OtherPerson and the READs first clears Boss before clearing OtherPerson. This happens because neither program actually keeps track of which email is being processed in next and instead just removes the max value regardless of if it is being processed or not.
- Next & Read Spam
 - This section tests what the program does when presented with a file that spans next and reads.
 - Example used
 - EMAIL Boss,Please don't spam read,12-05-2001
 - EMAIL Peer,Don't do it!,05-04-2002
 - COUNT
 - NEXT / READ
 - NEXT / READ
 - NEXT / READ
 - NEXT / READ
 - COUNT
 - Both programs do not crash when given this file but they do differ. ChatGPT simply ignores the invalid commands and prints no message to the user indicating nonsense commands. Gemini does print a message to the user but in the case of NEXT the message is unclear as it says that a READ command has to be between NEXTs instead of just saying that the current

email must be read before going to the next email or something similar. This is the same way vice versa with spammed READ commands. Both programs are incorrect here but Gemini only slightly so.

- Empty Box
 - This section tests if the program handles next and read commands when there are no emails in the heap
 - Example used
 - COUNT
 - NEXT
 - READ
 - COUNT
 - Exact same situation as Next & Read Spam, ChatGPT ignores while Gemini prints an error message. This time the error message is informative as it tells the user that there are no emails. Therefore, only ChatGPT is incorrect and Gemini is correct in this instance.

Conclusion

Both programs have issues that prevent them from being correct. Gemini has a list-based implementation using an array that fits the instructions while ChatGPT uses a pointer that is satisfactory but only technically so. Both of their heap functions lack the proper protections to prevent bad input. The comparison between emails of Gemini lacks a tie-breaker that breaks the heap format while ChatGPT has a comparison that in a tie creates a backwards order. Both programs can handle out of place and strange NEXT and READ usage but Gemini is informative whereas ChatGPT is silent. And finally, both programs fail to ensure that the email that NEXT printed is actually the one removed from the heap when READ is called. For which is technically more correct, I favor Gemini since the lack of proper messaging from ChatGPT about file errors is more damaging to the user experience than emails that are improperly sort, a sin both are guilty of even if Gemini is worse.

Execution Time

The execution time of both programs is very similar and has few problems. The biggest thing of note is that the Gemini program with its array implementation does not worry itself with freeing up memory space and clearing allocations unlike the ChatGPT implementation with its pointer-based design. Therefore, with both of these programs being similar besides that area, Gemini has a better execution time than the ChatGPT program.

Space Complexity

The space complexity of these two programs varies a lot due to one issue. While there are small issues such as Gemini using a `can_read` and `can_next` variable versus ChatGPT which just uses one variable `next_pending`, the biggest point of difference is in how emails are stored. Gemini simply creates an array of size `MAX_EMAILS` where `MAX_EMAILS` is a defined constant while ChatGPT uses a pointer that reallocates space whenever the array is going to breach its capacity. This means that ChatGPT can support high stores of emails while also keeping its size small when there are only a few emails. This is opposed to Gemini which has to allocate all of the space instantly and will remain large with both small and high email counts. This massive difference means that ChatGPT is far less space complex than Gemini.

Maintainability

The maintainability of both programs is challenged. The amount of comments in both programs are far below the amount of comments that should be in the program considering how large they are. The lack of line-by-line comments makes the readability of the programs difficult. However, I found the Gemini code to be far more readable due to its far more conventional design. Gemini implements most of its heap functions recursively whereas ChatGPT implements most of them in loops. The recursive definitions are far more common, so Gemini is more readable because of it. Additionally, ChatGPT leans heavily on using pointer and pointer manipulation which is unfamiliar to anyone who is not well versed in C. These differences create a huge difference in readability which in turn make a huge difference in maintainability. Both program also fail to properly handle functions being called with improper conditions such as an upheap being called with an empty heap. Overall, Gemini is more maintainable than ChatGPT but only slightly so.

Choice of Template

The choice came down to maintainability and execution time versus better space complexity for Gemini and ChatGPT respectively. In the end, I chose Gemini as the syntax used in ChatGPT was strange enough to create a serious problem with maintainability. Even with strong comments, there was no way that the ChatGPT program would become easy to understand or modify.

Change List

This section will list every change I made to the final program and the reason for the change.

- Correct category priority assignment in getCategoryValue
 - This improves correctness since it allows proper comparison to occur between emails.
- Add id value in emails for comparison in compareEmails
 - This improves correctness since it ensures that emails with same date and category will be properly placed before the
- Remove check for “Important Person” in getCategoryValue
 - This improves execution time and correctness since the program assumes good category names and thus has no need for this meaningless check.
- Add print message for if email cannot be added in insertEmail
 - This improves correctness since the user will now be informed if an email was not actually added.
- Replace can_next and can_read with current_email
 - This change makes it so when NEXT is called, the list of email extracts the max email and assigns it to current_email. Count will now add 1 to count if current_email is not null and READ only checks if current_email is null or not before it processes. This improves correctness since the program can now manage the surprise email situation from before.
- Add comments to everything
 - Adding comments increases readability which in turn increases maintainability.